Practical 10
Spring Boot + JPA CRUD Operations

Aim

To develop CRUD-based web applications using Spring Boot and JPA for managing Student and Employee records efficiently through REST APIs and database connectivity.

Learning Objectives

1. To understand the fundamentals of Spring Boot framework.
2. To learn the implementation of CRUD operations using JPA.
3. To understand RESTful API development in Spring Boot.
4. To connect Spring Boot applications with MySQL database.
5. To understand layered architecture using Controller, Service, Repository, and Entity classes.
6. To perform database operations such as Create, Read, Update, and Delete.
7. To learn Object Relational Mapping (ORM) using JPA.
8. To test APIs using Postman or browser requests.

Software Requirements

- Java JDK 17 or above
- Spring Boot
- Maven
- MySQL Database
- IntelliJ IDEA / Eclipse IDE
- Postman / Web Browser

Theory

Spring Boot is a Java-based framework used to create standalone and production-ready applications quickly and efficiently. It reduces configuration complexity and provides built-in support for web applications, REST APIs, and database integration.

JPA (Java Persistence API) is used for managing relational data in Java applications. It provides ORM (Object Relational Mapping), which maps Java classes to database tables. Spring Data JPA simplifies database operations by providing predefined methods such as `save()`, `findAll()`, `findById()`, and `deleteById()`.

In this practical, two CRUD applications are implemented:

- Student Management System
- Employee Record Management System

Both applications use:

- Entity Classes to represent database tables
- Repository Layer for database interaction
- Service Layer for business logic
- Controller Layer for handling HTTP requests

REST APIs are used to perform CRUD operations:

- `POST` → Create records
- `GET` → Retrieve records
- `PUT` → Update records
- `DELETE` → Remove records

Spring Boot automatically configures the application and integrates seamlessly with MySQL database, making development faster and easier.

11.1. To develop a Student Management System using Spring Boot and JPA for performing CRUD (Create, Read, Update, Delete) operations on student records.

Input:
Application.properties

```
server.port=8082
spring.datasource.url=jdbc:mysql://localhost:3306/management_db?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=Shriyash@11
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

StudentController.java

```
package com.shriyash.mca.controller;

import com.shriyash.mca.entity.Student;
import com.shriyash.mca.service.StudentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StudentController {

    @Autowired
    private StudentService studentService;

    @GetMapping
    public List<Student> getAllStudents() {
        return studentService.getAllStudents();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
        return studentService.getStudentById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
```

```
    @PostMapping
    public Student createStudent(@RequestBody Student student) {
        return studentService.saveStudent(student);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Student> updateStudent(@PathVariable Long id, @RequestBody Student
studentDetails) {
        try {
            return ResponseEntity.ok(studentService.updateStudent(id, studentDetails));
        } catch (RuntimeException e) {
            return ResponseEntity.notFound().build();
        }
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteStudent(@PathVariable Long id) {
        studentService.deleteStudent(id);
        return ResponseEntity.ok().build();
    }
}
```

Student.java

```
package com.shriyash.mca.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String course;

    public Student() {}
```

```
    public Student(String name, String email, String course) {
        this.name = name;
        this.email = email;
        this.course = course;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

StudentRepository.java

```
package com.shriyash.mca.repository;

import com.shriyash.mca.entity.Student;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

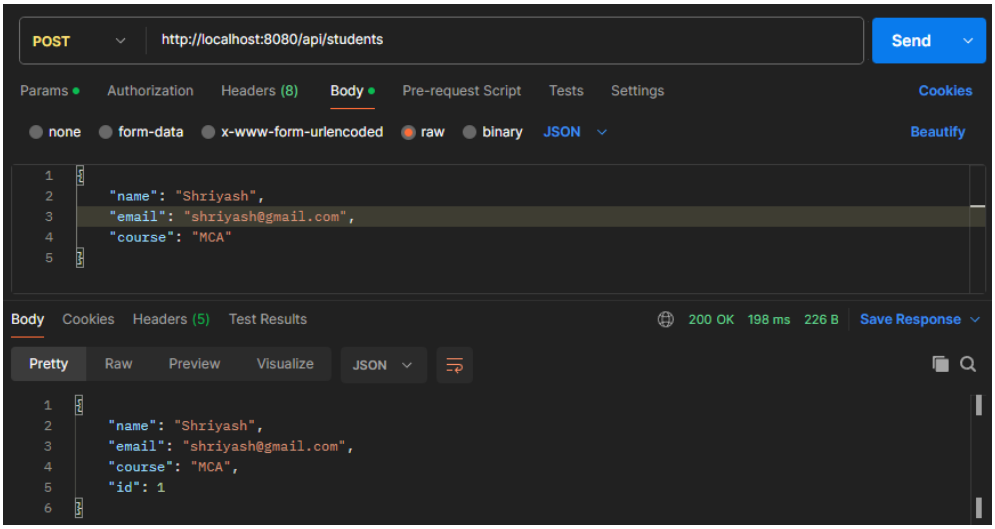
StudentService.java

```
package com.shriyash.mca.service;

import com.shriyash.mca.entity.Student;
import com.shriyash.mca.repository.StudentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class StudentService {

    @Autowired
    private StudentRepository studentRepository;

    public List<Student> getAllStudents() {
        return studentRepository.findAll();
    }

    public Optional<Student> getStudentById(Long id) {
        return studentRepository.findById(id);
    }

    public Student saveStudent(Student student) {
        return studentRepository.save(student);
    }

    public void deleteStudent(Long id) {
        studentRepository.deleteById(id);
    }

    public Student updateStudent(Long id, Student studentDetails) {
        Student student = studentRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Student not found for id :: " + id));
```

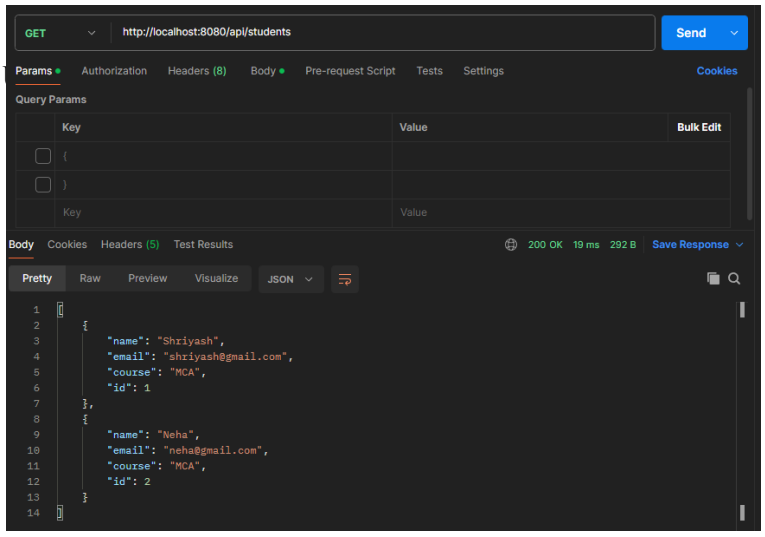
```
        student.setName(studentDetails.getName());
        student.setEmail(studentDetails.getEmail());
        student.setCourse(studentDetails.getCourse());

        return studentRepository.save(student);
    }
}
```

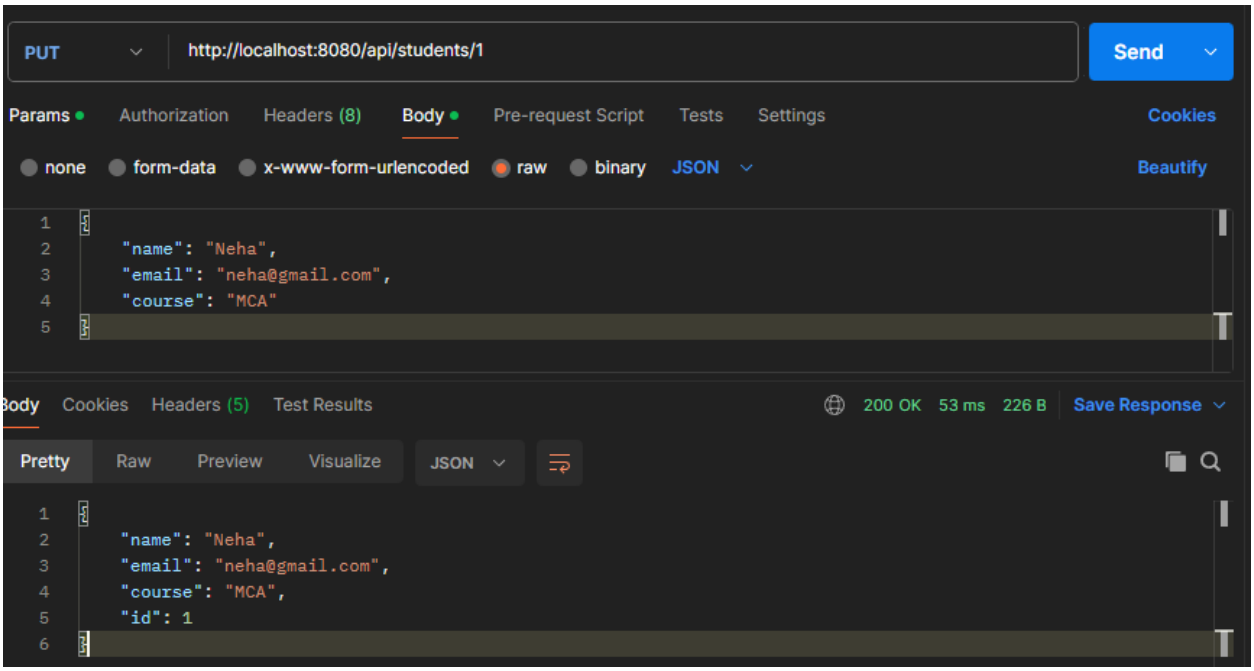
Output:

Insert Data:

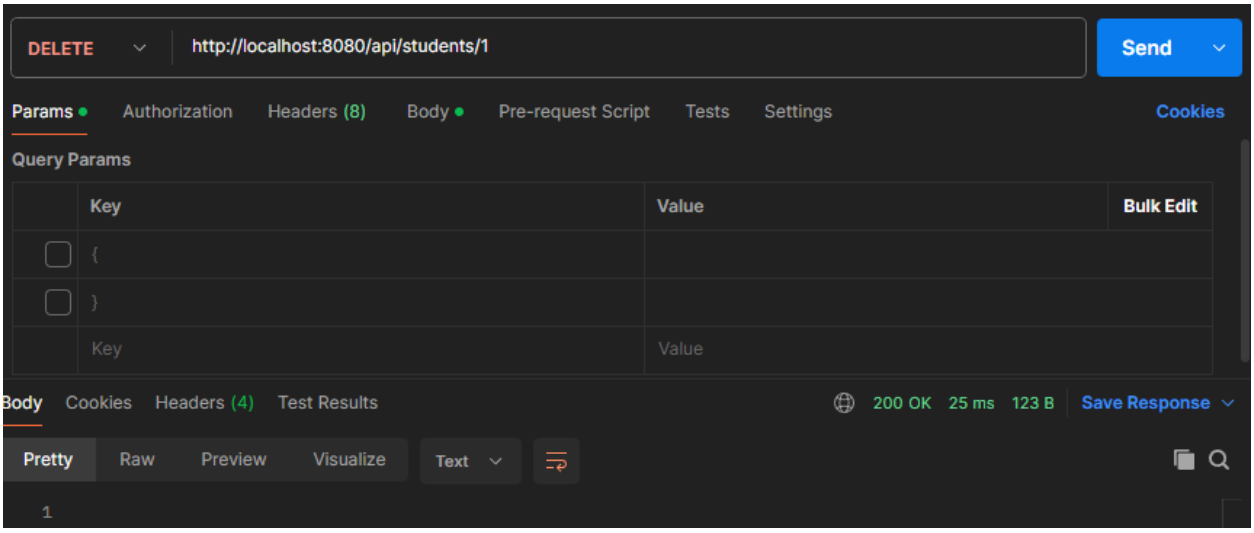


View Data:





Delete Data:



11.2.

Input:To develop an Employee Record Management System using Spring Boot and JPA for managing employee records through CRUD operations.

EmployeeController.java

```
package com.shriyash.mca.controller;

import com.shriyash.mca.entity.Employee;
import com.shriyash.mca.service.EmployeeService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @Autowired
    private EmployeeService employeeService;

    @GetMapping
    public List<Employee> getAllEmployees() {
        return employeeService.getAllEmployees();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployeeById(@PathVariable Long id) {
        return employeeService.getEmployeeById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public Employee createEmployee(@RequestBody Employee employee) {
        return employeeService.saveEmployee(employee);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Employee> updateEmployee(@PathVariable Long id, @RequestBody
Employee employeeDetails) {
        try {
            return ResponseEntity.ok(employeeService.updateEmployee(id, employeeDetails));
        } catch (RuntimeException e) {
```

```
            return ResponseEntity.notFound().build();
        }
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteEmployee(@PathVariable Long id) {
        employeeService.deleteEmployee(id);
        return ResponseEntity.ok().build();
    }
}
```

Employee.java

```
package com.shriyash.mca.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String department;
    private Double salary;

    public Employee() {}

    public Employee(String name, String department, Double salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
```

```
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public Double getSalary() {
        return salary;
    }

    public void setSalary(Double salary) {
        this.salary = salary;
    }
}
```

EmployeeRepository.java

```
package com.shriyash.mca.repository;

import com.shriyash.mca.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
}
```

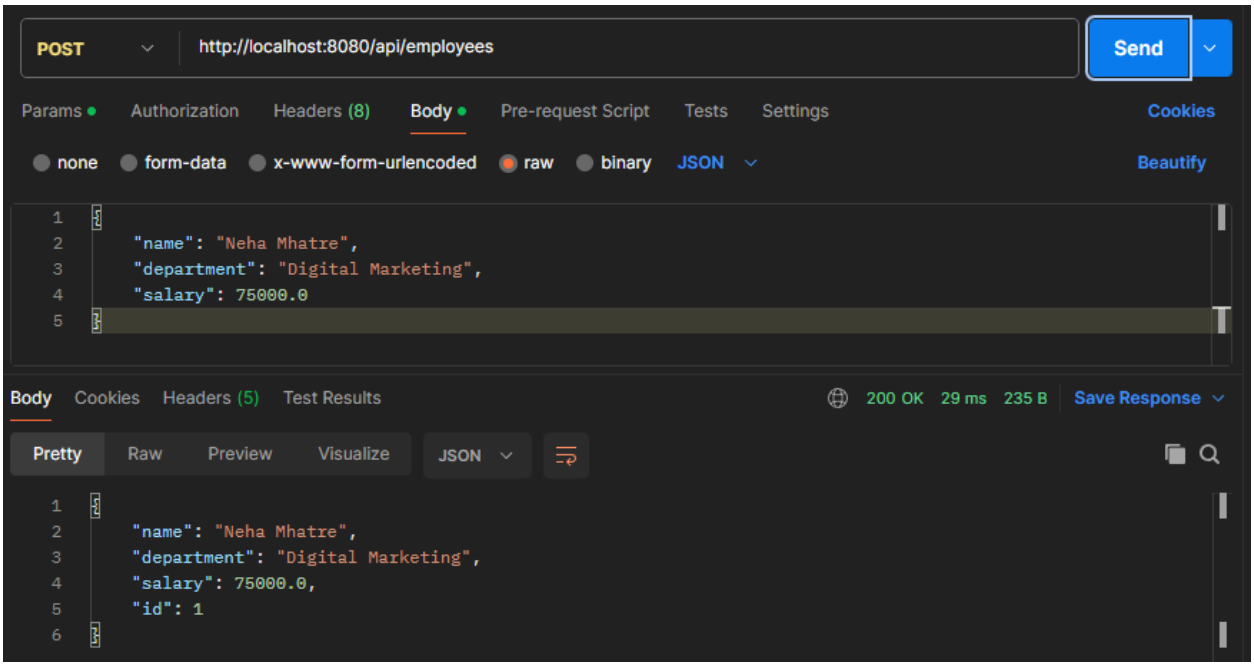
EmployeeService.java

```
package com.shriyash.mca.service;

import com.shriyash.mca.entity.Employee;
import com.shriyash.mca.repository.EmployeeRepository;
```

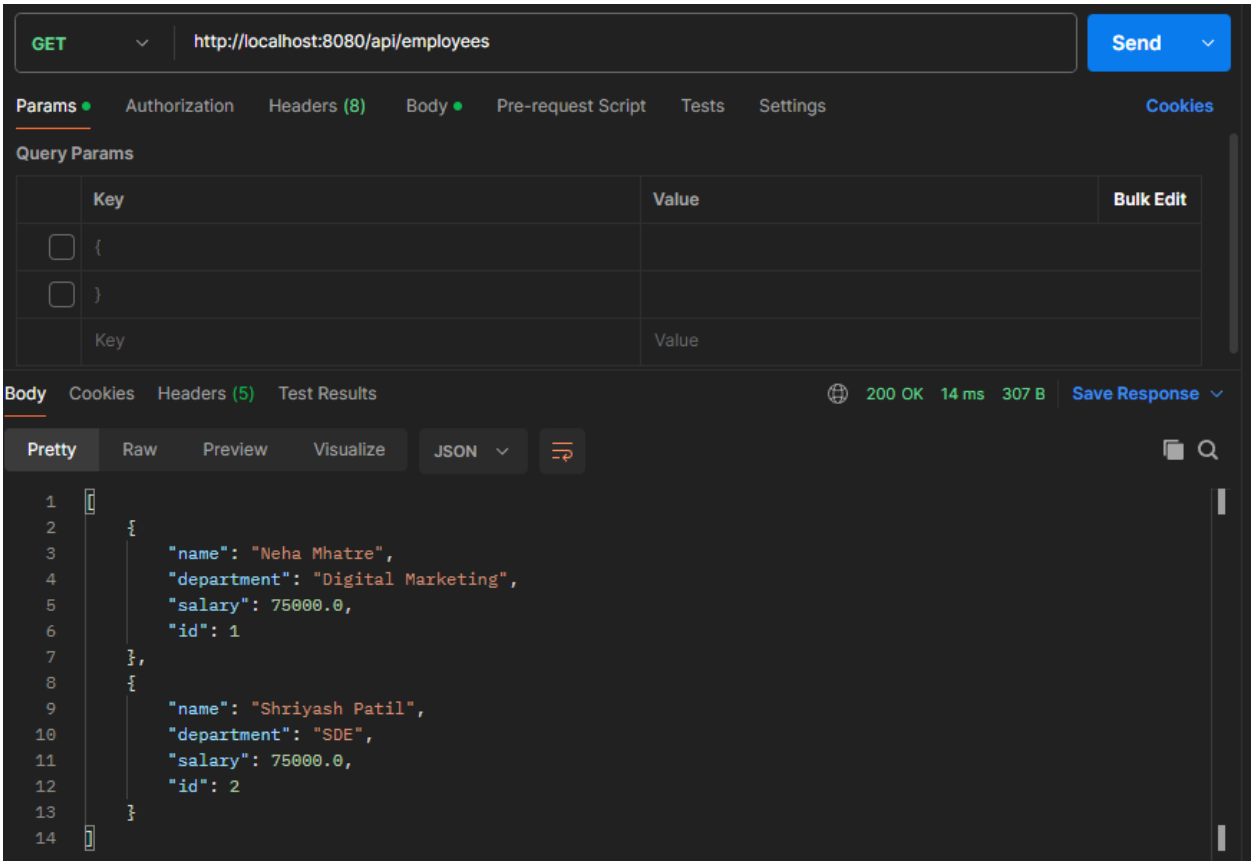
```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

    public List<Employee> getAllEmployees() {
        return employeeRepository.findAll();
    }

    public Optional<Employee> getEmployeeById(Long id) {
        return employeeRepository.findById(id);
    }

    public Employee saveEmployee(Employee employee) {
        return employeeRepository.save(employee);
    }

    public void deleteEmployee(Long id) {
        employeeRepository.deleteById(id);
    }

    public Employee updateEmployee(Long id, Employee employeeDetails) {
        Employee employee = employeeRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Employee not found for id :: " + id));

        employee.setName(employeeDetails.getName());
        employee.setDepartment(employeeDetails.getDepartment());
        employee.setSalary(employeeDetails.getSalary());

        return employeeRepository.save(employee);
    }
}
```

Output:

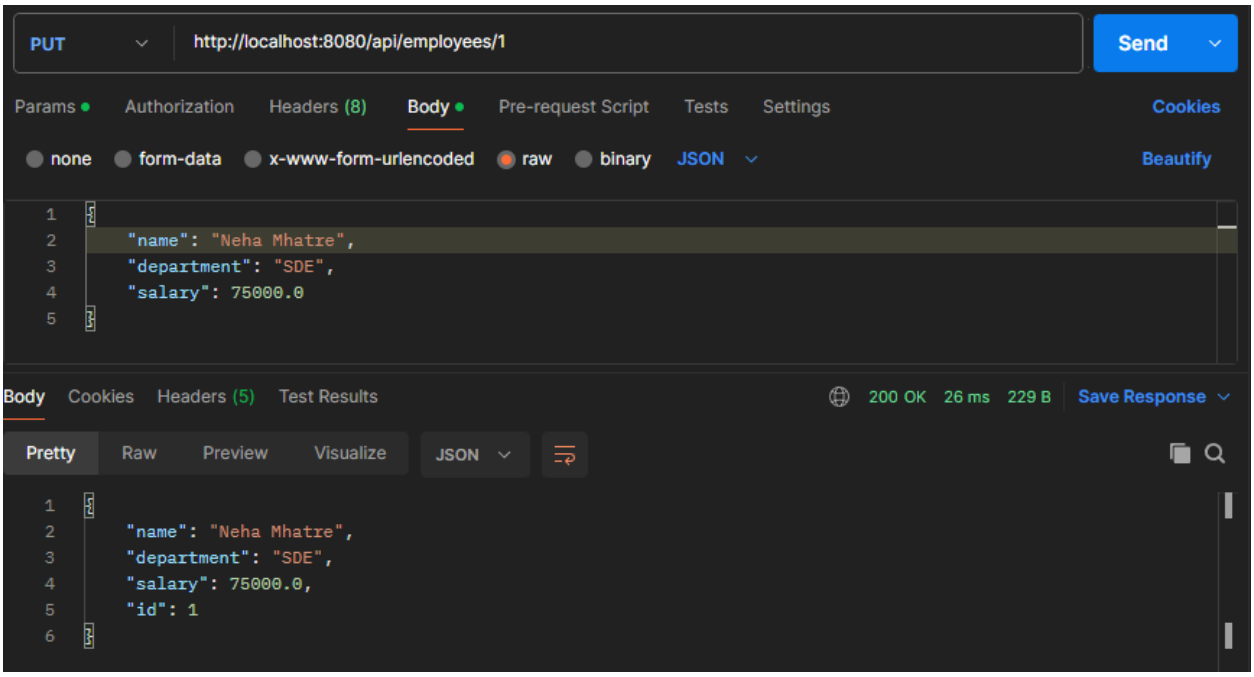
Insert Data:



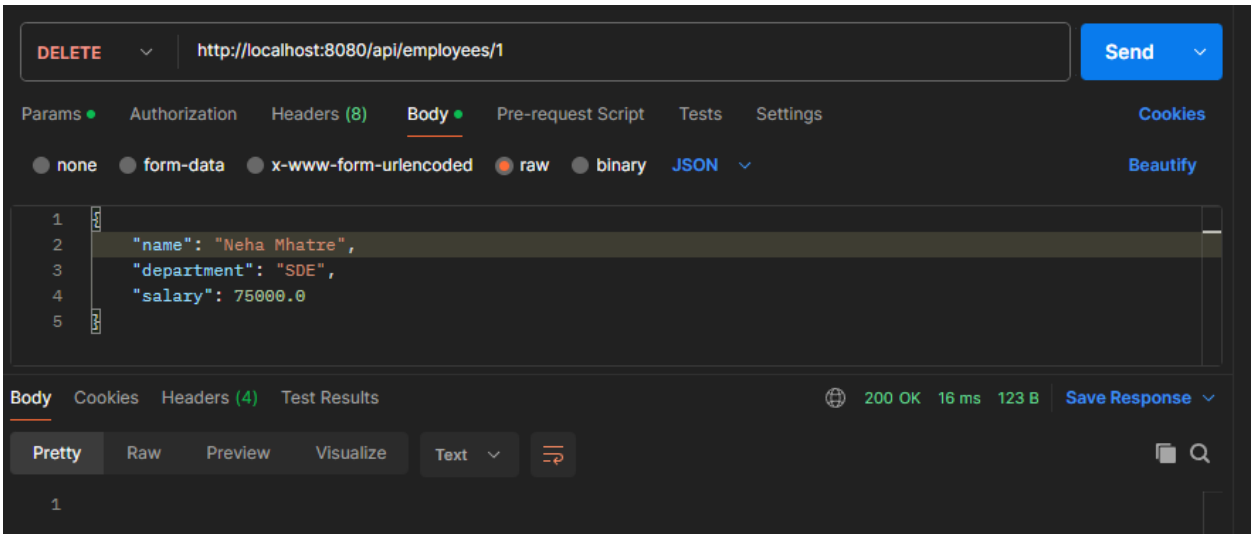
View Data:



Update Command:



Delete Data:



Conclusion

The implementation of CRUD operations using Spring Boot and JPA successfully demonstrated how modern Java-based web applications can efficiently manage database records through REST APIs. The Student Management System and Employee Record Management System provided practical exposure to creating, reading, updating, and deleting records using Spring Data JPA and MySQL database integration.